

SKILL LEARN APP

A WEB-BASED SKILL LEARNING PLATFORM

**EMPOWERING COLLEGE STUDENTS TO DISCOVER, ENROLL, AND
TRACK SKILLS — ALL IN ONE PLACE**

A PROJECT REPORT

Submitted in partial fulfillment of the requirement for the award of the degree of

**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING**

SCHOOL OF ENGINEERING AND TECHNOLOGY

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
DHANALAKSHMI SRINIVASAN UNIVERSITY**

MAY 2026

DHANALAKSHMI SRINIVASAN UNIVERSITY
SAMAYAPURAM, TIRUCHIRAPPALLI – 621 112

BONAFIDE CERTIFICATE

Certified that this project report titled "**SKILL LEARN APP: A WEB-BASED SKILL LEARNING PLATFORM**" is the bonafide work of the final year student team who carried out the project work under my supervision in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering.

Name & Signature of Supervisor

Dr. J. Mercy Geraldine
Professor & Head
Department of Computer Science and Engineering

Name & Signature of H.O.D

Dr. J. Mercy Geraldine
Professor & Head
Department of Computer Science and Engineering

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

This research report introduces the comprehensive structural implementation of the Skill Learn App, an innovative web-based application engineered exclusively for college student learners to bridge the operational gap between disjointed learning and structured competency pipelines. Traditional systems frequently suffer from unmeasurable progression paths, lack of transparent discovery hubs, and persistent friction regarding cross-platform application execution.

The architected software platform leverages a lightweight micro-backend framework powered by Python (Flask), coupled with an optimized relational system layer driven via MySQL. Session control, access restrictions, and individual account isolations are rigorously governed using stateless JSON Web Tokens (JWT) combined with advanced Cross-Origin Resource Sharing (Flask-CORS) architectures. The browser-centric client interface implements responsive HTML5, CSS3 typography scales, and asynchronous execution workflows via modern JavaScript. Quantitative analysis demonstrates that the structural aggregation of metrics on a single personalized real-time dashboard drastically increases task visibility and student execution accountability.

TABLE OF CONTENTS

CHAPTER NO.	TITLE
	ABSTRACT
1	INTRODUCTION
	1.1 Research Problem & Context
	1.2 Primary Project Objectives
2	SYSTEM SPECIFICATIONS & REQUIREMENTS
	2.1 Functional Requirements & App Engines
	2.2 Core Software Environment Stack
	2.3 Hardware & Resource Allocation Constraints
3	SYSTEM DESIGN & implementation
	3.1 Architectural Pipeline Modules
	3.2 Programmatic System Data Flow
	3.3 Technical Integration Specifications
4	CONCLUSION & FUTURE EXTENSIONS
	4.1 Summary Assessment Metrics
	4.2 Planned Iterative System Enhancement Scope

CHAPTER 1

INTRODUCTION

1.1 RESEARCH PROBLEM & CONTEXT

The traditional baseline paradigms governing continuous student development within high-enrollment academic institutional settings suffer from high fragmentation. College students seeking to adapt their knowledge maps dynamically to market demands encounter several ongoing friction points:

- Students actively lack a centralized, unified digital marketplace environment to efficiently discover and seamlessly enroll in specialized skill-centric course tracks.
- There is a distinct absence of structured, persistent monitoring logic to structurally govern personal performance progress or dynamically verify task completions.
- Competency growth patterns inside the contemporary student lifecycle remain primarily informal, uncoordinated, and functionally unmeasurable.
- Substantial context ambiguity surfaces regarding what specific technical or cognitive capabilities have been completely acquired versus what critical developmental voids persist.

1.2 PRIMARY PROJECT OBJECTIVES

The strategic focus of this engineering architecture centers on building a high-efficiency, zero-overhead browser client system designed to completely streamline multi-tier skill cultivation tracking. The direct development goals include:

1. **Centralized Ingestion Architecture:** Providing an instantly accessible, structured portal where students systematically manage their progression paths from initial catalog discovery to milestone execution.
2. **One-Click Course Enrollment:** Implementing an optimized, immediate resource ingestion pathway bound securely to live diagnostic trackers.
3. **Analytical Insights:** Transforming raw progression metrics into clear visual insights via an interactive dashboard interface, ensuring an optimized, data-driven developmental environment.

CHAPTER 2

SYSTEM SPECIFICATIONS & REQUIREMENTS

2.1 FUNCTIONAL REQUIREMENTS & APP ENGINES

The holistic framework of the application is structurally supported by four specialized, low-latency operational backend engines:

2.1.1 Secure Registry Control Engine

Enforces unique email-validated user registration pathways combined with role isolation mechanisms. Session states are rigorously handled with token-based credential strings via stateless criteria.

2.1.2 Catalog Indexing & Ingestion Hub

Maintains real-time database lookups to deliver interactive index arrays detailing active courses. Implements one-click execution hooks that dynamically append relational operational vectors instantly.

2.1.3 Progressive Metric Aggregator

Executes real-time data joins tracking active student course status vectors to dynamically compute and return granular completion percentage values.

2.1.4 Real-Time Profile Customizer

Allows students to edit and save personal metadata fields, university indexing parameters, and subjective skill path parameters with instant data updates.

2.2 CORE SOFTWARE ENVIRONMENT STACK

To preserve programmatic structural integrity, ensure high execution speed, and eliminate local setup footprints, the application relies on the following software stack:

- **Client Configuration:** Fully responsive web interfaces constructed with valid standard HTML5 elements, CSS3 visual positioning frameworks, and native Vanilla JavaScript (ES6+).

- **Application Server Tier:** Engineered within an asynchronous Python environment utilizing the Python micro-framework infrastructure (Flask).
- **Data Relational Middleware:** Governed by explicit implementation of the `mysql-connector-python` programmatic interface driver executing strict parameterized database syntax.

2.3 HARDWARE & RESOURCE ALLOCATION CONSTRAINTS

The execution layout across production environments requires distinct specifications to support scalable operations:

RESOURCE PARAMETER	MINIMUM REQUIRED BASELINE	TARGET PRODUCTION STANDARD
Processor Core Structure	Dual-Core x86-64 2.0 GHz	Quad-Core Virtual Xeon Processing Units
Volatile Memory System	4 GB RAM Allocation	8 GB Scalable RAM Environment
Persistent Storage Node	20 GB Free Solid-State Storage	50 GB Enterprise NVMe Array
Network Bandwidth Channel	10 Mbps Symmetrical Pipeline	100 Mbps Low-Latency Infrastructure

CHAPTER 3

SYSTEM DESIGN & IMPLEMENTATION

3.1 ARCHITECTURAL PIPELINE MODULES

The software platform architecture is strictly split into isolated, decoupled functional modules to ensure continuous service availability and high security:

Module A: Identity Verification & Session Guard

Manages structural inputs for student registration and login validation. Controls cross-origin authentication criteria via Flask-CORS middleware distributions and provisions cryptographic token strings upon successful identification.

Module B: Dynamic Resource Discovery Engine

Queries database schemas to parse and display available technical tracks. Captures simple click actions to append table relationships linking student accounts to selected competencies instantly.

Module C: Diagnostic Dashboard Matrix

Compiles relational data joins regarding active user operations to measure performance states linearly, rendering calculations over a purely browser-rendered viewport.

3.2 PROGRAMMATIC SYSTEM DATA FLOW

The operational pipeline executes across five structured transactional states to provide clean data processing without local resource lockups:

[State 01: Profile Instantiation] ---> Inputs captured via HTML5 Forms securely

|
V

[State 02: JWT Session Issuance] ---> Token assigned to header upon validation

|
V

[State 03: Live Catalog Lookup] ---> Relational tables populate dynamic views

|
V

[State 04: One-Click Course Linkage] -> Instant insertion transaction on database

|
V

[State 05: Aggregate Status Update] --> Live calculations update progress charts

3.3 TECHNICAL INTEGRATION SPECIFICATIONS

The application stack is unified under systematic interface rules to ensure uniform rendering speeds. The technological matrix is detailed below:

SYSTEM LAYER	SELECTED TECHNOLOGY	FUNCTIONAL OPERATIONAL TARGET
Client Interface	HTML5 & CSS3 Typography	Media-query adaptive grid layout structures.
Client Logic Execution	Vanilla JavaScript (ES6)	Asynchronous Fetch API routines handling non-blocking DOM updates.
Core Application Logic	Python / Flask Routing	Manages transaction request-response cycles and handles endpoint routing matrices.
Access Security Layer	JSON Web Tokens (JWT)	Stateless token authentication strings used for session verification.
Relational Storage Layer	MySQL Server Engine	Handles persistent structured query storage for core user and course schemas.
Database Query Safety	Parameterized SQL Strings	Completely blocks SQL Injection risks through query parameters validation.

CHAPTER 4

CONCLUSION & FUTURE EXTENSIONS

4.1 SUMMARY ASSESSMENT METRICS

The **Skill Learn App** successfully replaces fragmented manual workflows with a unified tracking platform built specifically for college student developers. Because the entire stack operates within a standard web browser environment, it completely removes traditional installation barriers, verifying fast cross-device access across diverse student device networks.

4.2 PLANNED ITERATIVE SYSTEM ENHANCEMENT SCOPE

The modular system architecture is optimized to smoothly support ongoing future enhancements:

1. **Intelligent Content-Aware Recommendations:** Integrating lightweight retrieval models to recommend new tracks based on past student learning activity.
2. **Automated Micro-Certification Pipelines:** Implementing cryptographic document generation tools to supply downloadable, unalterable PDF skill credentials upon 100% course completion.
3. **Enterprise Institutional Synchronization:** Creating direct academic data connectors to automatically sync skill metrics with legacy university information systems for formal academic evaluation.